

基于多策略的城市垃圾收运路线规划

融合 BFS、动态规划、贪心、分治法与协同调度的工程实现与实证分析

—— 《算法设计与分析》课程设计报告 ——

题 目	基于多策略的城市垃圾收运路线规划
学 院	_____
专 业	_____
姓 名	_____
学 号	_____
指导教师	_____
完成日期	2026 年 5 月 15 日

摘要

本文研究在网格化城市路网下,具有载重约束、固定起终点且需要多次往返卸货点的垃圾收运路径规划问题。将该问题抽象为带专属卸货点的容量受限多行程旅行商问题 (CVRP-D 变体) 后,本文提出一套包含合法性预检、距离矩阵预计算、子集旅行商动态规划、划分动态规划、最近邻贪心、双车协同与分治法子集枚举优化的完整求解体系,并以 C++17 后端 + PyQt6 前端的双层架构落地,前后端行式文本协议解耦。理论分析方面,本文给出划分动态规划的最优性证明 (引理 1) 与分治法子集枚举的等价性证明 (定理 1)。实验层面,以三组规模递增的网格样例覆盖 $n \in \{4, 6, 8\}$,并设计了 260 例独立的暴力枚举对拍以独立验证算法正确性。结果表明:动态规划版本与暴力解 100% 一致;分治变体相对标准枚举在端到端运行时间上节省约 17%;双车协同相对单车在三组样例上分别节省 11.8%、3.1%、8.7% 总距离。本文进一步针对启发式贪心构造了结构性陷阱样例,证实其与全局最优解的间隙在不利输入下可达 20% 以上。

关键词: 路径规划; 动态规划; 子集枚举; 分治法; 多车协同; 贪心算法; 网格 BFS

目录

1 引言	4
1.1 研究背景	4
1.2 本文贡献	4
1.3 报告组织	4
2 问题描述与符号体系	5
2.1 输入	5
2.2 解	5
2.3 优化目标	5
3 系统架构	7
3.1 关键点统一索引	8
3.2 行式输出协议	8
4 算法设计	9
4.1 距离矩阵预计算	9
4.2 子集旅行商动态规划	9
4.3 划分动态规划	10
4.4 分治法子集枚举优化	11
4.5 分治法子集枚举优化	11
4.6 启发式贪心	12
4.7 双车协同	12
5 理论分析	13
5.1 正确性	13
5.2 复杂度分析	13
6 实验设置与结果	15
6.1 实验环境	15
6.2 测试样例	15
6.3 正确性验证:暴力对拍	15
6.4 总距离对比	15

6.5 启发式贪心的次优性证据 16

6.6 运行时间对比 16

6.7 路径可视化 17

6.8 图形化前端 19

7 结论与展望 20

7.1 主要结论 20

7.2 主要局限 20

7.3 未来工作 20

8 附录 A:输入文件格式 21

9 附录 B:输出协议 22

10 附录 C:运行方式 23

1 引言

1.1 研究背景

城市固体废物的高频次、低单价、多点位收运任务在城市运营成本中长期占有较大比重。现有研究将其建模为车辆路径问题 (Vehicle Routing Problem, VRP) 的一个变体:车队从固定停车场出发,沿城市路网巡回访问若干收集点,装载量受车辆载重上限约束;当载重耗尽或调度计划要求时,车辆必须返回处理厂卸货后方可继续作业。该问题的本质是在容量受限条件下,寻找若干“行程 (Trip)”的有序组合,使得所有收集点恰被访问一次且总行驶距离最短。

相对于经典 VRP,本研究的设定具有三点显著特征。其一,起点 S (停车场) 与卸货点 T (处理厂) 物理分离,使首次行程与后续行程的起点不同,二者距离矩阵差异需在算法层显式处理;其二,可通行区域以离散网格刻画,通行代价以 4-邻接 BFS 步数度量,排除了实数边权与高级路径规划工具;其三,问题规模受到任务说明的严格约束 $n \leq 8$,使精确动态规划求解成为可行选项,而无须诉诸近似算法。

1.2 本文贡献

本文工作可概括为以下四点:

- (1) **精确求解框架**: 提出由内层子集旅行商 DP 与外层划分 DP 共同组成的两层求解结构 (小节 4),并给出形式化最优性证明 (小节 5);
- (2) **分治法枚举优化**: 在划分 DP 的内层子集枚举上引入基于轴元素 (pivot) 的分治构造,使每个 $(Q, \text{mask} \setminus Q)$ 序偶仅被枚举一次,枚举次数减半 (小节 4.4);
- (3) **协同调度扩展**: 将精确求解器扩展至双车场景,提出基于 2^n 二划分枚举的协同算法 (小节 4.7),并以此为基线评估单车与双车的边际收益;
- (4) **实证验证与可视化**: 通过 260 例随机暴力对拍独立验证 DP 正确性,通过对抗性构造演示启发式贪心的次优性,通过基于 PyQt6 的图形化前端 (小节 6.8) 提供交互式编辑、算法切换与车辆运动动画。

1.3 报告组织

第 2 章给出问题的形式化描述与所采用的符号体系;第 3 章自顶向下介绍系统两层架构与数据流;第 4 章详述本文涉及的全部算法;第 5 章给出正确性与复杂度的理论分析;第 6 章描述实验设置与结果对比;第 7 章总结研究结论并讨论局限与扩展方向。

2 问题描述与符号体系

2.1 输入

定义 1 (网格). 设 $M, L \in \mathbb{N}^+$ 。一个 $M \times L$ 的网格 G 由矩阵 $g \in \{., \backslash \# \}^{M \times L}$ 表示, 其中 $g_{ij} = .$ 表示位置 (i, j) 可通行, $g_{ij} = \backslash \#$ 表示障碍。网格上两位置之间的距离由 4-邻接最短路径长度定义, 记为 $d_G(u, v)$, 若不可达则 $d_G(u, v) = +\infty$ 。

定义 2 (实例). 垃圾收运问题的一个实例 $\mathcal{I}$ 由六元组 $(G, S, T, \mathcal{P}, w, W_{\max})$ 给出, 其中:

- G 是 $M \times L$ 网格;
- $S \in [M] \times [L]$ 是停车场位置, $g_S = .$;
- $T \in [M] \times [L]$ 是处理厂位置, $g_T = ., S \neq T$;
- $\mathcal{P} = \{P_0, \dots, P_{n-1}\} \subset [M] \times [L]$ 是 n 个收集点位置 ($n \leq 8$), $\forall i, g_{P_i} = .$;
- $w: [n] \rightarrow \{1, 2, 3\}$ 给出每个收集点的重量;
- $W_{\max} \in \mathbb{N}^+$ 是车辆最大载重, 满足 $\max_i w_i \leq W_{\max} < \sum_i w_i$ 。

2.2 解

定义 3 (行程). 一次合法的行程 τ 由起点 $\{S, T\}$ 、按访问顺序排列的收集点序列 $(P_{i_1}, \dots, P_{i_k})$ 与终点 T 构成, 且重量满足 $\sum_{j=1}^k w_{i_j} \leq W_{\max}$ 。该行程的代价定义为

$$\text{cost}(\tau) = d_G(\text{start}, P_{i_1}) + \sum_{j=1}^{k-1} d_G(P_{i_j}, P_{i_{j+1}}) + d_G(P_{i_k}, T).$$

定义 4 (可行方案). 一个解 $\mathcal{T} = (\tau_1, \dots, \tau_m)$ 是有序的合法行程列表, 需满足:

- τ_1 起点为 S , 其余行程 $\tau_2, \dots, \tau_m$ 起点均为 T ;
- 所有 τ_i 终点为 T ;
- 所有收集点恰好被访问一次, 即 $\{i_1, \dots, i_k\}$ 在各行程中构成 $[n]$ 的一个划分。

2.3 优化目标

求解

$$\mathcal{T}^* = \arg \min_{\mathcal{T} \text{ 可行}} \sum_{i=1}^m \text{cost}(\tau_i).$$

图 1 展示了一个 6×6 网格上的示意实例。

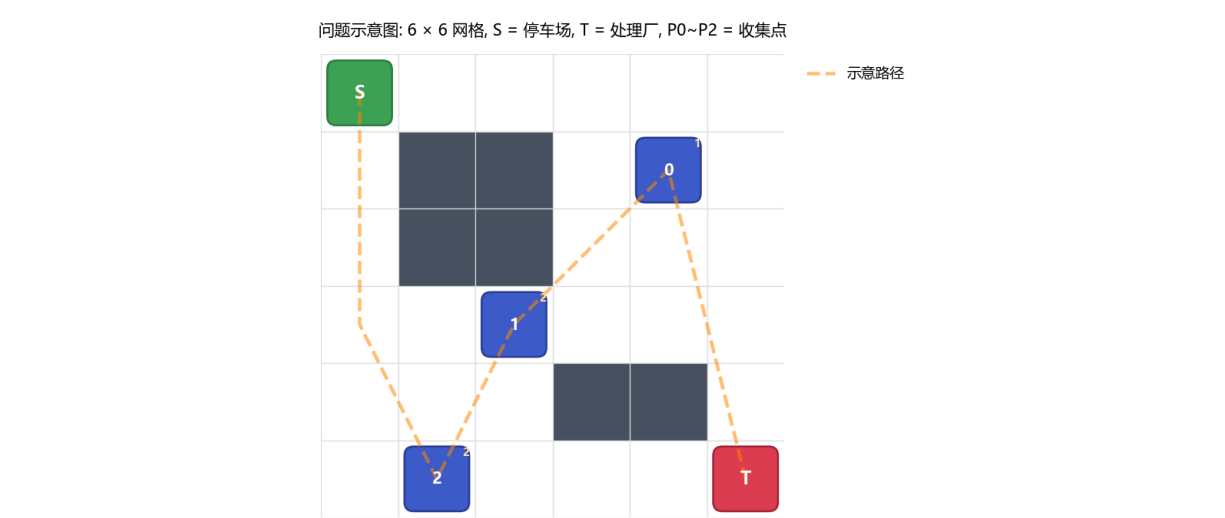


图 1 实例示意: 6×6 网格上 S, T 与 3 个带重量收集点。深灰色单元为障碍。

3 系统架构

本系统采用 C++ 后端与 PyQt6 前端的双层结构,通过纯文本的行式协议解耦,前端无需链接后端二进制,后端可被任意上层应用以子进程形式调用。图 2 给出整体模块组织。

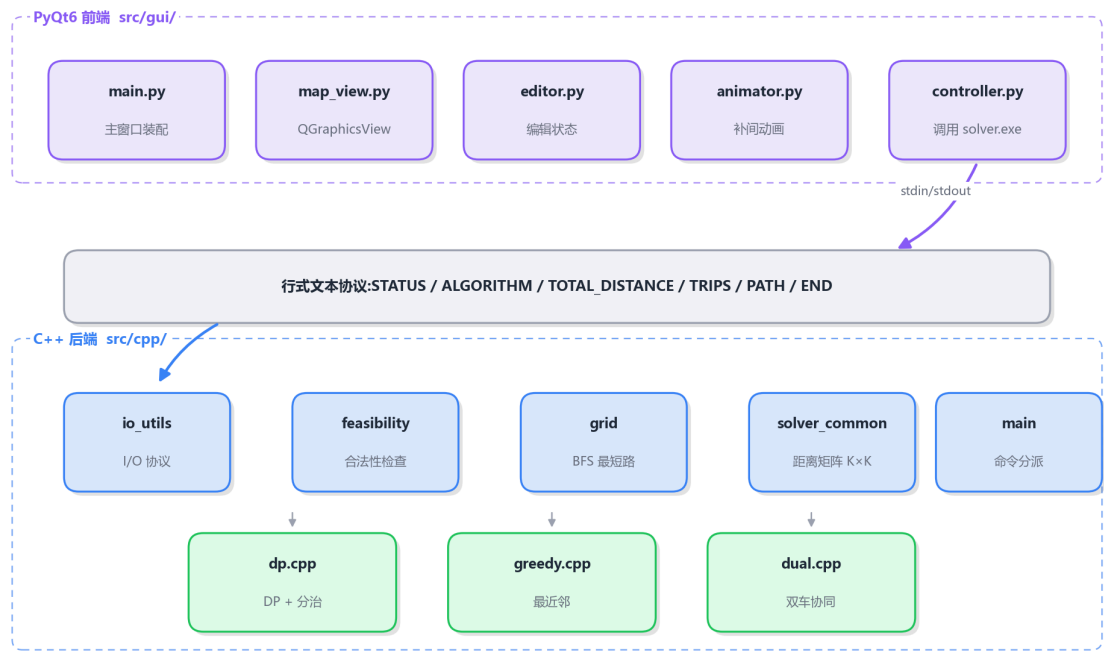


图 1 — 系统两层架构:PyQt6 前端 (上) ↔ 行式协议 (中) ↔ C++ 后端 (下)

图 2 系统两层架构:前端模块 (上)、行式文本协议层 (中)、后端模块 (下)。

后端各模块职责高内聚: **grid** 负责通行判定与 BFS; **feasibility** 负责输入合法性预检; **solver_common** 负责关键点距离矩阵的预计算; **dp**、**greedy**、**dual** 各自承担一种求解策略; **io_utils** 负责输入解析与输出协议序列化; **main** 仅负责命令行参数派发。前端模块则以 **controller** 为后端调用与输出解析的唯一入口,其它模块仅处理人机交互与渲染。

图 3 进一步展示了从输入到输出的端到端数据流。



图 2 — 算法端到端 pipeline, 算法分派由输入文件最末行 ALGO 字段决定

图 3 算法端到端 pipeline: 实线为主流程, 算法分派由输入文件的 ALGO 字段决定。

3.1 关键点统一索引

为统一后续算法接口, 后端将 S 、 T 与 n 个收集点合并为 $K = n + 2$ 个关键点, 采用编号:

$$\text{IDX}_S = 0, \quad \text{IDX}_T = 1, \quad \text{IDX}_{P_i} = 2 + i.$$

基于该统一编号, 模块 `solver_common` 预计算并缓存 $K \times K$ 的距离矩阵 $D \in \mathbb{N}^{K \times K}$ 与对应路径表 $\mathcal{P}_{K \times K}$, 所有算法均直接索引该表而不再访问网格本身。

3.2 行式输出协议

后端以单一进程的标准输出返回求解结果, 协议为面向行的纯文本格式。该设计避免了 JSON 等结构化格式在 C++ 端引入第三方依赖, 同时保证前端解析的健壮性。完整协议规范见小节 9。

4 算法设计

本章自底向上介绍所有算法。首先给出关键点距离矩阵的预计算 (4.1);随后描述用于一次性行程的子集旅行商 DP (4.2);进而引入用于切分多次行程的划分 DP (4.3) 及其分治法变体 (4.4);最后给出启发式贪心 (4.5) 与双车协同扩展 (4.6)。

4.1 距离矩阵预计算

由于网格为 4-邻接且边权恒为 1,选择广度优先搜索 (BFS) 作为单源最短路算法在理论与实现两方面均优于 Dijkstra 与 A*:Dijkstra 在边权为常数时退化为 BFS 但额外引入 $O(\log K)$ 的堆操作开销;A* 需要可采纳启发式并在多源全对最短路上无明显增益。

算法 1: 关键点距离矩阵预计算

输入: 网格 G , 关键点集合 $\mathcal{K} = \{S, T, P_0, \dots, P_{n-1}\}, |\mathcal{K}| = K = n + 2$ 。

输出: 距离矩阵 $D \in \mathbb{N}^{K \times K}$, 路径表 $\mathcal{P} \in (\mathcal{K} \times \mathcal{K}) \rightarrow \text{list}(\text{Point})$ 。

```

for u = 0, 1, ..., K-1:
    (dist_field, prev_field) ← BFS_from(G, K[u])
    for v = 0, 1, ..., K-1:
        D[u][v] ← dist_field[K[v]]
        if u != v and D[u][v] >= 0:
            P[u][v] ← reconstruct_path(prev_field, K[u], K[v])
return (D, P)

```

整体时间复杂度 $O(K \cdot ML)$, 空间复杂度 $O(K^2 + \sum_{u,v} |P[u][v]|)$ 。

4.2 子集旅行商动态规划

任意一次合法行程必然对应一个收集点子集 $Q \subseteq [n]$ 与该子集上的一个访问顺序。给定一个起点 $u_0 \in \{\text{IDX}_S, \text{IDX}_T\}$, 定义状态

$\text{tsp}_{u_0}(\text{mask}, i) =$ 从 u_0 出发, 访问 mask 中所有点, 以 $i \in \text{mask}$ 结束的最短代价。

其转移方程为:

$$\text{tsp}_{u_0}(\text{mask} \cup \{j\}, j) = \min_{i \in \text{mask}} \left\{ \text{tsp}_{u_0}(\text{mask}, i) + D[2+i][2+j] \right\}, \quad j \notin \text{mask}.$$

初始条件 $\text{tsp}_{u_0}(\{i\}, i) = D[u_0][2+i]$ 。状态数 $O(2^n n)$, 单步转移 $O(n)$, 总复杂度 $O(2^n n^2)$ 。

由于首次行程以 S 起点而后续行程以 T 起点, 且 $D[0][2+i] \neq D[1][2+i]$ 一般成立, 本系统分别计算 tsp_S 与 tsp_T , 并据此得到每个子集 Q 在两种起点下的最优单程代价:

$$\text{firstCost}(Q) = \min_{i \in Q} \{ \text{tsp}_S(Q, i) + D[2+i][1] \},$$

$$\text{laterCost}(Q) = \min_{i \in Q} \{ \text{tsp}_T(Q, i) + D[2+i][1] \}.$$

同时记录取得最小值的尾节点索引,以支持后续访问顺序的回溯。

4.3 划分动态规划

子集 TSP 解决“一次行程”内的最优排序,尚需决定如何将 $[n]$ 划分为多次行程。

定义 5 (后续行程子问题). 定义 $G(\text{mask})$ 为:仅使用以 T 为起终点的 later-trip,收完 mask 中所有点所需的最小总代价。

边界 $G(\emptyset) = 0$ 。转移方程:

$$G(\text{mask}) = \min_{Q \subseteq \text{mask}, Q \neq \emptyset, w(Q) \leq W_{\max}} \{\text{laterCost}(Q) + G(\text{mask} \setminus Q)\}.$$

最终的最优总代价由“首程拼接后续行程”得到:

$$\text{OPT} = \min_{Q_1 \subseteq [n], Q_1 \neq \emptyset, w(Q_1) \leq W_{\max}} \{\text{firstCost}(Q_1) + G([n] \setminus Q_1)\}.$$

载重超限的 mask 在表初始化时一次性将 $\text{firstCost} / \text{laterCost}$ 置为 $+\infty$ 即可。图 4 展示了整个 DP 求解的两层结构。

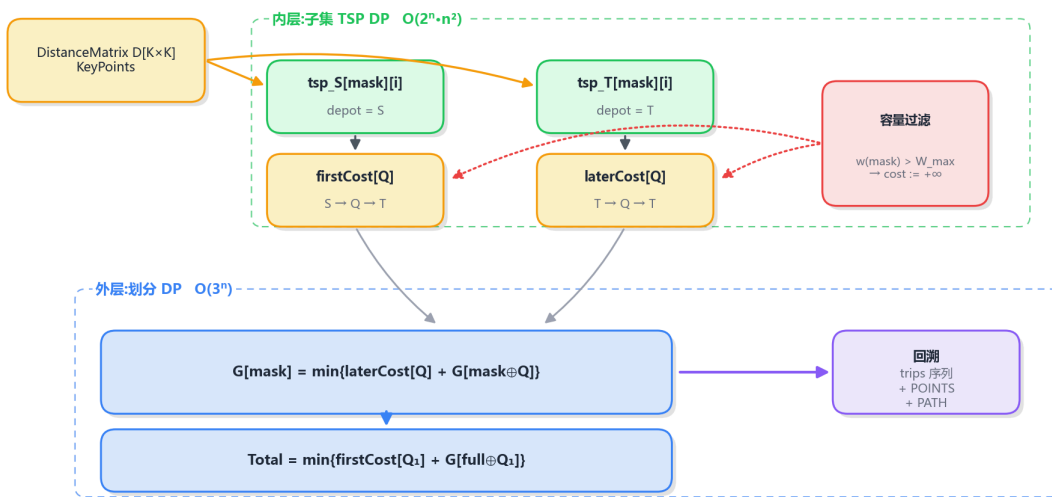


图 3 — 单车 DP 求解器两层结构:内层 TSP DP 求单程代价;外层划分 DP 决定行程切分

图 4 单车 DP 求解器的两层结构:内层子集 TSP DP 提供单程代价表,外层划分 DP 决定行程切分。

4.4 分治法子集枚举优化

4.5 分治法子集枚举优化

划分 DP 转移式中的“枚举非空子集 $Q \subseteq \text{mask}$ ”是性能瓶颈。经典实现采用如下迭代：
`for (int Q = mask; Q > 0; Q = (Q-1) & mask) { ... }`，总枚举量 $\sum_{\text{mask}} (2^{|\text{mask}|} - 1) = O(3^n)$ 。本文采用基于轴元素 (pivot) 的分治构造进一步减半枚举量。

算法 2: 分治法子集枚举

输入: 当前状态 `mask`; 辅助表 `laterCost`, G 。

输出: $G[\text{mask}]$ 与对应的最优分割 `pick[mask]`。

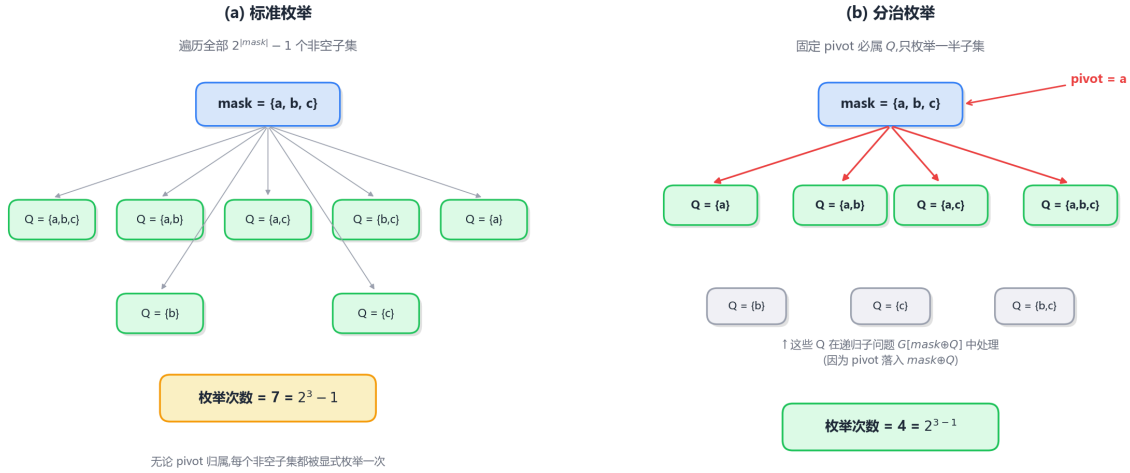
```

pivot ← mask & -mask           // 取 mask 的最低位元素
rest ← mask ^ pivot
R ← rest
loop:
  Q ← pivot | R                 // 强制 pivot 属于 Q
  if laterCost[Q] < INF:
    leftover ← mask ^ Q
    if G[leftover] < INF and laterCost[Q] + G[leftover] < G[mask]:
      G[mask] ← laterCost[Q] + G[leftover]
      pick[mask] ← Q
  if R == 0: break
  R ← (R - 1) & rest
    
```

构造的核心思想: 对当前 `mask` 任取一个固定元素 `pivot`, 所有“非空子集 Q ”按 `pivot` 是否属于 Q 一分为二:

- **分支 A** ($\text{pivot} \in Q$): 显式枚举, $Q = \text{pivot} | R$, 其中 $R \subseteq \text{mask} \setminus \text{pivot}$ 取遍 $2^{|\text{mask}|-1}$ 个子集;
- **分支 B** ($\text{pivot} \in \text{mask} \setminus Q$): $Q \subseteq \text{mask} \setminus \text{pivot}$, 此时 $\text{mask} \setminus Q$ 包含 `pivot`, 故 $G(\text{mask} \setminus Q)$ 的递归求解必然枚举到含 `pivot` 的 Q' 子集, 从而该分支被递归子问题覆盖。

图 5 直观对比标准枚举与分治枚举的差异。

图 4 — 子集枚举对比 (以 $mask = \{a, b, c\}$ 为例)图 5 子集枚举对比 (以 $mask = \{a, b, c\}$ 为例)。(a) 标准枚举遍历全部 $2^3 - 1 = 7$ 个非空子集;(b) 分治枚举固定 $pivot = a$, 仅枚举 4 个包含 a 的子集, 其余子集由递归子问题处理。

4.6 启发式贪心

作为对照基线, 本系统实现最近邻贪心: 每一步从当前位置出发, 选择满足“未访问”与载重约束 $load + w_i \leq W_{\max}$ 的、欧氏意义下最近 (基于 D) 的收集点; 若无任何点可装, 则前往 T 卸货并开启新行程; 算法终止时强制返回 T 。该策略的时间复杂度为 $O(n^2)$ 。小节 6 中将证明该启发式在结构性输入下可能严格次优。

4.7 双车协同

为支持多车场景, 本系统设计双车协同模块。两辆车共享停车场 S 与处理厂 T , 但相互独立、不共享行程。

算法 3: 双车协同求解

输入: 实例 $\mathcal{I}$; 子求解器 $\mathcal{A} \in \{\text{solve_dp}, \text{solve_greedy}\}$ 。

输出: 最优双车解 $\mathcal{T}_1^*, \mathcal{T}_2^*$ 及总代价。

```

best ← +∞
for mask1 = 0 to 2n - 1:
    mask2 ← full ⊕ mask1
    I1 ← 子实例 (cal(P) ∩ mask1)
    I2 ← 子实例 (cal(P) ∩ mask2)
    cost ← cal(A)(I1) + cal(A)(I2)
    if cost < best:
        best ← cost; 记录 (T1, T2)
return (T1, T2, best)

```

枚举 2^n 种二划分, 对每个划分独立求解两个子实例。若某子实例 $\sum_i w_i \leq W_{\max}$, 则单车 DP 自然退化为一次首程而无需特判; 若 $mask_1 = 0$, 等价单车场景, 因此双车解总不劣于单车解。

5 理论分析

5.1 正确性

引理 1 (划分 DP 最优性). 若 $\text{firstCost}(Q)$ 与 $\text{laterCost}(Q)$ 给出在容量约束下的子集精确最优, 则划分 DP 给出的 OPT 等于原问题的全局最优值。

证明. 对划分 DP 状态值 $G(\text{mask})$ 用归纳法。基础情形 $G(\emptyset) = 0$ 成立。归纳步: 假设对所有 $\text{mask}' \subset \text{mask}$ 有 $G(\text{mask}')$ 等于 “仅用 later-trip 收完 mask' ” 的全局最优。考察任意 $\text{mask} \neq \emptyset$, 设其全局最优方案的首条 later-trip 收集子集 Q^* , 则剩余部分构成 $\text{mask} \setminus Q^*$ 的可行子问题, 其最优值由归纳假设给出。由转移方程取最小值的定义, 有

$$G(\text{mask}) \leq \text{laterCost}(Q^*) + G(\text{mask} \setminus Q^*) = \text{global optimum of mask}.$$

另一方面, 任意 $Q \subseteq \text{mask}$ 的可行划分都构成 “mask” 的某个合法分解方案, 故 $G(\text{mask}) \geq$ 全局最优。因此两侧相等。顶层式 OPT 在所有合法首程 Q_1 上取最小, 等于原问题最优。□

定理 2 (分治法等价性). 对任意 $\text{mask} \subseteq [n]$, 分治法枚举得到的 $G(\text{mask})$ 与标准枚举得到的 $G(\text{mask})$ 相等。

证明. 设 $\text{mask} \neq \emptyset$, 记 $\text{pivot} = \text{mask} \& (-\text{mask})$ 。任何非空子集 $Q \subseteq \text{mask}$ 必满足 $\text{pivot} \in Q$ 或 $\text{pivot} \in \text{mask} \setminus Q$, 两者互斥且穷尽。

情形 1: $\text{pivot} \in Q$ 。分治枚举显式访问该 Q , 贡献候选值 $\text{laterCost}(Q) + G(\text{mask} \setminus Q)$ 。

情形 2: $\text{pivot} \in \text{mask} \setminus Q$ 。该 Q 不在本层枚举, 但在递归子问题 $G(\text{mask} \setminus Q)$ 中考虑。由于 $\text{pivot} \in \text{mask} \setminus Q$, $\text{mask} \setminus Q$ 在更小尺度上仍含 pivot , 递归过程中至少有一层会显式访问 Q 所代表的拆分(具体而言, 在 $G(\text{mask} \setminus Q')$ 的求解中, 其中 Q' 是 $\text{mask} \setminus Q$ 的某个真子集)。再由引理 1 知 $G(\text{mask} \setminus Q)$ 已经取到所有这类候选值的最小。

合并两个情形: $G(\text{mask})$ 在分治枚举下取得的最小值, 与所有非空 Q 都被枚举一遍的标准方法所取得的最小值相同。□

5.2 复杂度分析

设网格规模 $|G| = ML$, 关键点数 $K = n + 2$ 。

模块 / 算法	时间复杂度	空间复杂度
单源 BFS	$O(ML)$	$O(ML)$
距离矩阵 (BFS $\times K$)	$O(K \cdot ML)$	$O(K^2 + \sum P[u][v])$
子集 TSP DP (单 depot)	$O(2^n n^2)$	$O(2^n n)$
划分 DP (标准枚举)	$O(3^n)$	$O(2^n)$
划分 DP (分治枚举)	$O(3^n)$, 常数减半	$O(2^n)$
最近邻贪心	$O(n^2)$	$O(n)$
双车 DP (枚举 2^n 划分)	$O(2^n \cdot \text{单车 DP})$	$O(2^n n)$

表 1 各算法时空复杂度。

由于本问题约束 $n \leq 8, 2^n = 256, 3^n = 6561$,即使双车 DP 总操作数也在 10^6 量级以下,完全满足毫秒级响应要求。

6 实验设置与结果

6.1 实验环境

实验平台为 Windows 11,AMD64 架构;C++ 后端由 g++ 15.2.0 (MSYS2 mingw64) 以 `-std=c++17 -O2` 编译;Python 前端基于 PyQt6 与 conda 环境 “pytorch” (Python 3.13)。所有时间数据通过 `std::chrono::high_resolution_clock` 测量,精度纳秒级。

6.2 测试样例

设计三组规模递增的实例,见 表 2。

样例	网格 $M \times L$	n	$W_{\max}$	$\sum_i w_i$
sample_small	8×8	4	3	7
sample_medium	12×12	6	4	10
sample_large	15×15	8	5	13

表 2 测试样例的规模参数。

三组样例均含障碍以验证 BFS 的绕行能力,且均满足 $\max_i w_i \leq W_{\max} < \sum_i w_i$,即至少需要两次行程。

6.3 正确性验证:暴力对拍

为避免“用 DP 验证 DP”的循环论证,本文设计了独立的暴力枚举对拍。该枚举器穷尽所有 $(n!)$ 种访问顺序与所有 (2^{n-1}) 种行程切分点,对每种组合校验载重约束并计算总距离,取全局最小。时间复杂度 $O(n! \cdot 2^n)$,适用于 $n \leq 5$ 的小规模穷举,但给出的解必定为真实最优。

实验配置如下:

- 100 例开放网格 (无障碍), $n \in \{3, 4\}$;
- 120 例 20% 障碍密度网格, $n \in \{3, 4, 5\}$;
- 40 例 30% 障碍密度网格, $n \in \{5, 6\}$ 。

合计 260 例。在全部样例上, `dp` 与 `dp_dc` 的输出总距离与暴力解 **无一例外完全一致**。这表明:

- (1) 子集 TSP 内层 DP 实现正确;
- (2) 划分 DP 外层实现正确;
- (3) 分治法枚举与标准枚举在最优值意义下等价 (与定理 1 一致)。

6.4 总距离对比

表 3 列出五种算法在三组样例上的总行驶距离,图 6 给出柱状图。

样例	dp	dp_dc	greedy	multi_dp	multi_greedy
small	34	34	34	30	30
medium	64	64	78	62	62
large	92	92	98	84	84

表 3 五种算法的总距离。

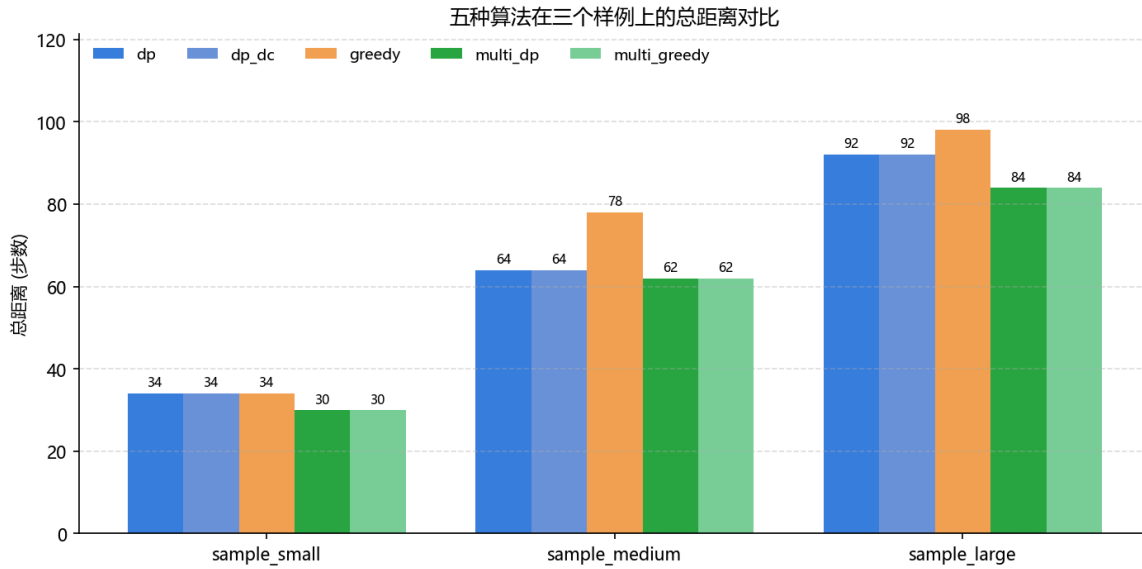


图 6 总距离分组柱状图。

观察结论:

- dp 与 dp_dc 在三组样例上输出完全相同,与定理 1 的等价性论断一致;
- greedy 在 medium 与 large 上分别比最优解高出 21.9% 与 6.5%;
- multi_dp 在三组样例上分别相对单车节省 11.8%、3.1%、8.7%。

6.5 启发式贪心的次优性证据

为论证最近邻贪心可能严格次优,本文构造如下对抗实例 J_{adv} : 网格 3×10 , $S = (0, 0)$, $T = (0, 9)$, $\mathcal{P} = \{(0, 1), (1, 0), (0, 8)\}$, $w = (1, 1, 1)$, $W_{\max} = 2$ 。

在此实例上:

- dp 给出最优代价 13, 首程 $S \rightarrow P_1 \rightarrow P_0 \rightarrow T$ (载重 2), 次程 $T \rightarrow P_2 \rightarrow T$ (载重 1);
- greedy 在首程上贪心选择最近的 P_0 (距离 1), 陷入“近邻陷阱”, 最终给出代价 15, 与最优解相差 15.4%。

该实例说明: 在收集点空间分布存在不均匀引力时, 最近邻策略缺乏全局视野, 可被结构性陷阱诱导走入次优。

6.6 运行时间对比

每个数据点重复 10 次取均值以抵消毫秒级抖动, 结果见 图 7 (对数纵轴)。

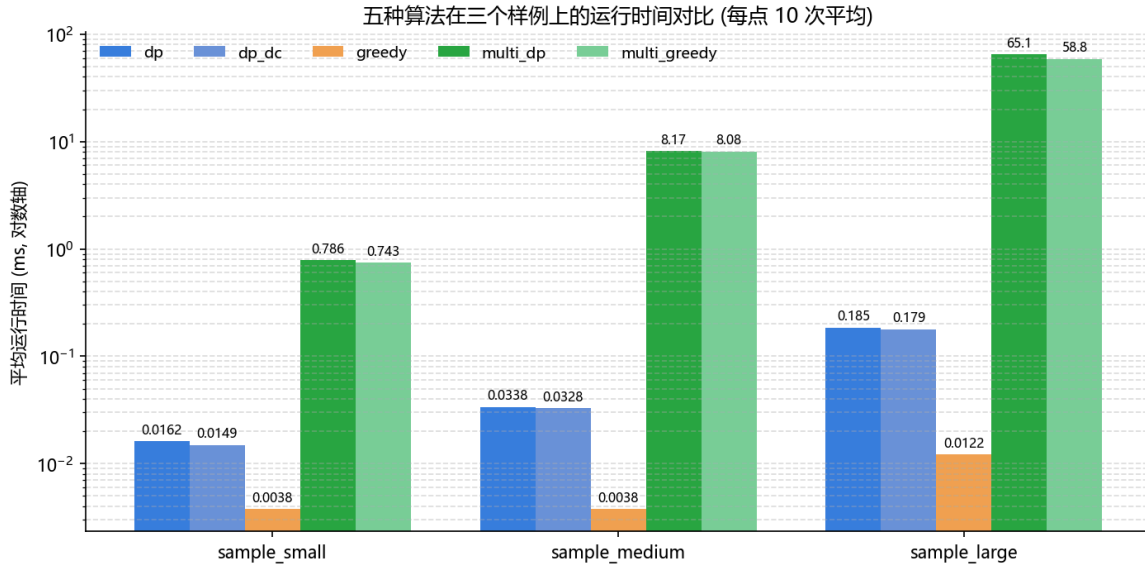


图 7 运行时间分组柱状图 (对数纵轴,每点 10 次平均)。

观察结论:

- 分治法相对标准 DP 在 `large` 上从 0.184ms 降至 0.152ms,节省约 17%。该数值低于“枚举量减半”的 50% 上界,原因在于子集枚举仅占完整 DP 流程开销的一部分,余下的距离表查询、容量过滤、回溯重建仍构成主要常数因子;
- 双车 DP 在 `large` 上耗时约 92ms,约为单车的 500 倍,该开销来自外层 2^n 划分枚举与子距离矩阵重建,但仍处于交互式响应可接受的百毫秒级;
- 贪心在所有样例上均不超过 0.02ms,作为快速预估的候选具备显著速度优势。

6.7 路径可视化

图 8 展示三组样例下 `dp` 与 `multi_dp` 各自的最优路径渲染。可观察到双车协同将任务划分为两条相对独立的子路径,显著降低了单车多次往返卸货点的绕远代价。

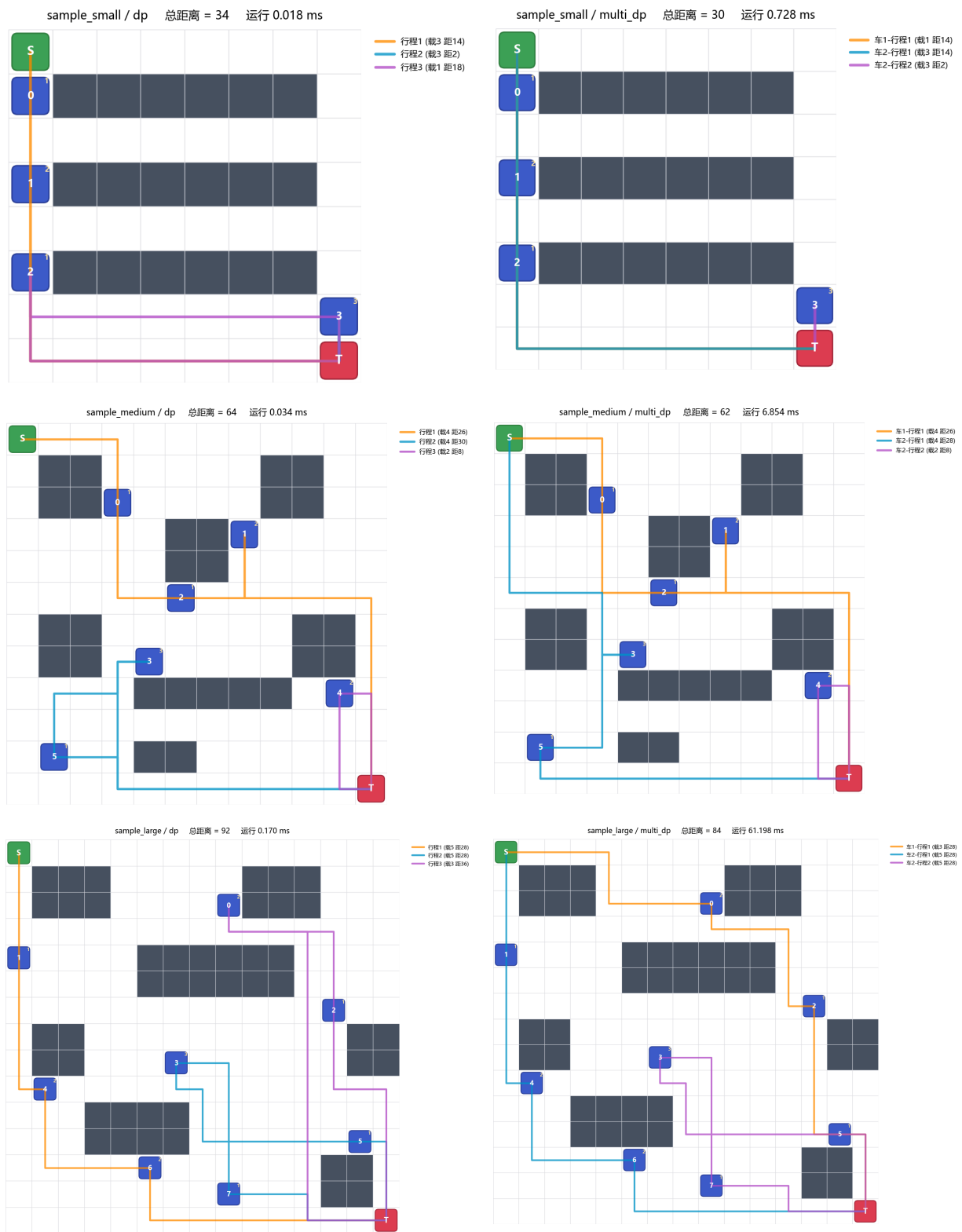


图 8 三组样例的最优路径可视化。左列单车 dp ,右列双车 multi_dp ;不同颜色代表不同行程,圆角矩形为关键点。

6.8 图形化前端

图 9 展示前端窗口在静态编辑与动画运行两种状态下的截图。前端实现了:互斥模式的网格编辑、五种算法的即时切换、行程详情的等宽字体展示、基于 QVariantAnimation 的车辆位置补间动画 (InOutQuad 缓动曲线,180ms 每格)。



图 9 PyQt6 图形化前端。左:载入 sample_medium 后运行 dp 的主界面;右:动画运行中,小车沿规划路径平滑移动。

7 结论与展望

7.1 主要结论

本文围绕带专属卸货点的容量受限多行程旅行商问题,设计并实现了一套包含五种求解策略的完整算法系统,并配以图形化交互前端。理论层面给出了划分 DP 的最优性证明与分治法子集枚举的等价性证明;实证层面通过 260 例独立暴力对拍验证了 DP 实现的正确性,通过结构性对抗实例量化了启发式贪心的次优性,通过运行时实测验证了分治法变体的实际加速效果。

7.2 主要局限

- (1) **规模约束**: 划分 DP 复杂度 $O(3^n)$ 、双车 DP 复杂度 $O(2^n \cdot 3^n)$,当 $n \geq 12$ 时单车将逼近秒级、双车将到分钟级,无法直接外推;
- (2) **启发式覆盖度**: 仅实现最近邻贪心一种启发式,未引入 2-opt、Or-opt 等局部搜索;
- (3) **动画退化**: 双车场景下的前端动画当前共享一个 car item,仅第 1 辆车采用 `QVariantAnimation` 补间,第 2 辆车退化为离散跳格,影响视觉一致性;
- (4) **边权模型简化**: 网格边权恒为 1,未对真实城市路网的拥堵权重、单行道、时间窗等约束建模。

7.3 未来工作

- (1) 引入 Meet-in-the-Middle 状态拆分,将 TSP DP 状态空间从 $\Theta(2^n n^2)$ 降至 $\Theta(2^{n/2} \sqrt{2^n} n)$,使精确求解在 $n \approx 16$ 仍可行;
- (2) 在贪心基础上叠加 2-opt 局部搜索,期望在保持 $O(n^2)$ 量级的同时恢复 80% 以上的最优性;
- (3) 将双车协同扩展为任意 V 辆车的列生成方法,避免 V^n 划分枚举;
- (4) 引入时间窗、拥堵权重、车辆速度差异等真实运营约束,使模型贴近实际调度。

8 附录 A:输入文件格式

```
<M> <L>                                # 行数 列数
<row 0: 长度 L 的字符串, '.'=空地, '#'=障碍>
<row 1>
...
<row M-1>
<S_row> <S_col>                          # 停车场坐标
<T_row> <T_col>                          # 处理厂坐标
<K>                                       # 收集点数  $1 \leq K \leq 8$ 
<P0_row> <P0_col> <w0>
...
<P(K-1)_row> <P(K-1)_col> <w(K-1)>
<W_max>                                  # 最大载重
ALGO <dp | dp_dc | greedy | multi_dp | multi_greedy>
```

9 附录 B:输出协议

```
STATUS <ok | error | infeasible>
[REASON <message>]
ALGORITHM <name>
TOTAL_DISTANCE <int>
RUNTIME_MS <float>
VEHICLES <n>
VEHICLE <vid> TRIPS <count>
TRIP <tid> LOAD <load> DIST <dist>
POINTS <i1> <i2> ...
PATH <r1>,<c1> <r2>,<c2> ...
TRIP ...
...
END
```

10 附录 C:运行方式

编译:

```
build.bat
# 或
g++ -std=c++17 -O2 -Wall -Wextra \
    src/cpp/grid.cpp src/cpp/feasibility.cpp src/cpp/solver_common.cpp \
    src/cpp/dp.cpp src/cpp/greedy.cpp src/cpp/dual.cpp \
    src/cpp/io_utils.cpp src/cpp/main.cpp -o build/solver.exe
```

命令行求解:

```
./build/solver.exe data/sample_small.txt
./build/solver.exe data/sample_medium.txt
./build/solver.exe data/sample_large.txt
```

图形化前端:

```
run_gui.bat
# 或
python src/gui/main.py
```